

Project Build Context — AI-Powered Nagpur Urban Crisis Management System

This file is the complete context for building this project. Read this entire file before writing any code.

1. Problem Statement

Heavy rainfall in Nagpur regularly causes waterlogging, road damage, and traffic congestion. Civic bodies respond reactively — after complaints come in or an area is already flooded. There is no unified, predictive system that combines weather, location, traffic, and citizen data to anticipate problems and route resources intelligently.

2. Proposed Solution (build exactly this — nothing more, nothing less)

An AI-powered Nagpur Urban Crisis Management System that uses rainfall, location, traffic, and citizen-report data to predict waterlogging and road problems, identify high-risk zones, suggest alternative routes, and prioritize civic action through a real-time dashboard.

Every feature built must map to one of these: **(1)** ingest rainfall/location/traffic/citizen data, **(2)** predict waterlogging/road-damage risk, **(3)** identify and display high-risk zones, **(4)** suggest safer routes, **(5)** prioritize civic action via a real-time dashboard.

Explicitly out of scope — do not build: live NMC data feed integration (no public source exists), real IoT hardware sensors (no physical devices available), native mobile app (use a responsive web app instead), NLP-based text classification of report descriptions, multi-language support, user social features. If a data source has no live/public feed, simulate it with a seed dataset instead of building a fake integration for it.

3. Finalized Tech Stack

Backend

- **Django + Django REST Framework (Python)** — all server logic, API, auth
- **PostgreSQL + PostGIS (Supabase)** — spatial database (zones, points, routes)
- **osmnx + NetworkX** — Nagpur road graph, A* algorithm for safe routing, edge weights penalized by zone risk score
- **Rule-based Python function for risk scoring** (not a trained ML model — realistic for the timeline; written as a swappable module)
- **Hugging Face Serverless Inference API** (pretrained vision models for pothole and waterlogging detection) — runs inference on citizen-submitted photos for two tasks: pothole detection and waterlogging-presence detection (yes/no, not depth estimation)
- **Twilio API (free trial tier)** — sends SMS/WhatsApp alerts when a zone's risk score crosses "High"
- **IMD public rainfall API (data.gov.in / IMD) / Open-Meteo** — real rainfall data instead of simulated, pulled on a schedule

Frontend

- **Next.js (JavaScript)** — App Router / Pages, server and client components (replacing React + Vite)
- **Leaflet.js** (OpenStreetMap tiles, dynamically loaded on client-side with `ssr: false`)
- **Tailwind CSS**, mobile-responsive (no separate native app)

Auth: Django's built-in auth, DRF `IsAdminUser` permission for admin-only routes

Deployment: Frontend → Netlify (Next.js Runtime / Adapter). Backend → Render. Database → Supabase (PostgreSQL + PostGIS).

Reasoning: Entire backend stays in Python (Django, risk logic, routing, CV inference calls via Hugging Face API) — no second language, deployed as a Docker container on Render with database managed via Supabase. Frontend utilizes Next.js for robust routing and rendering.

4. Data Model (Detailed Field-Level Schema Reference)

1. Zone (Wards / Geographic Boundaries)

Represents individual wards or administrative zones across Nagpur city.

Field	Type	Description
id	PrimaryKey	Unique ward/zone identifier.
name	CharField	Name of the ward (e.g., Dharampeth, Sitabuldi).
boundary	PolygonField (PostGIS)	Spatial boundary polygon outlining the geographic ward area.
elevation_factor	FloatField	Relative elevation indicator (identifies low-lying drainage basins).
drainage_capacity	FloatField	Stormwater and sewer runoff capacity rating.
dispatch_status	CharField (Choices)	Civic response tracking: <code>Unassigned</code> , <code>Dispatched</code> , or <code>Resolved</code> .

2. WeatherReading (Rainfall Monitoring)

Captures periodic rainfall measurements per zone from real weather APIs or simulations.

Field	Type	Description
id	PrimaryKey	Unique record identifier.
zone	ForeignKey (Zone)	Reference to the specific zone.
rainfall_intensity_mm	FloatField	Rainfall volume in millimeters (mm).
source	CharField (Choices)	Data origin: <code>imd_api</code> / live weather feed or <code>simulated</code> .
recorded_at	DateTimeField	Timestamp when the rainfall measurement was recorded.

3. TrafficReading (Road Congestion)

Tracks real-time traffic density and road delays per ward.

Field	Type	Description
id	PrimaryKey	Unique record identifier.
zone	ForeignKey (Zone)	Reference to the specific zone.
congestion_level	IntegerField (0–100)	Traffic index (0 = free-flowing traffic, 100 = gridlock).
recorded_at	DateTimeField	Timestamp when the congestion level was recorded.

4. Report (Citizen Incident Reports)

Citizen-submitted hazard reports processed via Hugging Face Vision AI.

Field	Type	Description
id	PrimaryKey	Unique report identifier.
reporter_location	PointField (PostGIS)	GPS coordinates (latitude, longitude) of the hazard.
photo	ImageField	Uploaded image of the road condition.
description	TextField	Citizen incident notes or commentary.
zone	ForeignKey (Zone)	Auto-assigned zone via spatial point-in-polygon lookup.
pothole_detected	BooleanField	True if Hugging Face AI confirms pothole presence.
pothole_confidence	FloatField (0.0–1.0)	Confidence score of the pothole detection model.
waterlogging_detected	BooleanField	True if Hugging Face AI confirms waterlogging presence.
waterlogging_confidence	FloatField (0.0–1.0)	Confidence score of the waterlogging detection model.
verification_status	CharField (Choices)	Administrative review state: Pending, Verified, or Rejected.
created_at	DateTimeField	Submission timestamp.

5. RiskScore (Dynamic Hazard Evaluation)

Aggregated computed crisis score calculated per zone based on live multi-source data.

Field	Type	Description
id	PrimaryKey	Unique computation identifier.
zone	ForeignKey (Zone)	Target zone reference.
score	FloatField (0–100)	Calculated weighted risk index.
category	CharField (Choices)	Classification level: Low (0-25), Medium (26-50), High (51-75), Severe (76-100).
computed_at	DateTimeField	Calculation timestamp.

6. AlertLog (Emergency Dispatches)

Audit trail of automated notifications triggered when zones reach High or Severe risk.

Field	Type	Description
id	PrimaryKey	Unique log identifier.
zone	ForeignKey (Zone)	Zone triggering the alert dispatch.
risk_category_at_send	CharField	Risk category at the moment the notification fired (High or Severe).
channel	CharField (Choices)	Delivery medium: SMS or WhatsApp (via Twilio API).
sent_at	DateTimeField	Dispatch timestamp.
status	CharField	Delivery status confirmation (e.g., Sent, Delivered, Failed).

5. Risk Scoring Formula

Risk Score (per zone) = $w_1 \times \text{Rainfall Intensity (last 3 hrs)} + w_2 \times \text{Drainage Capacity Deficit} + w_3 \times \text{Elevation Factor} + w_4 \times \text{Historical Incident Frequency} + w_5 \times \text{Current Citizen Report Density}$

Output categories: Low (0-25) | Medium (26-50) | High (51-75) | Severe (76-100)

Default weights: $w_1=0.35$, $w_2=0.25$, $w_3=0.15$, $w_4=0.15$, $w_5=0.10$. Tune only if time allows.

If a `Report` with `waterlogging_detected=True` exists for a zone, flag that zone's current `RiskScore` as *"photo-confirmed"* (display differently in UI — this is a validation layer on top of the formula, not a replacement for it).

6. API Contract

Endpoint	Method	Access	Purpose
<code>/api/reports/</code>	POST	Public	Citizen submits report (location + photo + description) → triggers Hugging Face detection automatically
<code>/api/reports/</code>	GET	Admin	List all reports
<code>/api/reports/{id}/verify/</code>	PATCH	Admin	Set <code>verification_status</code> to <code>Verified/Rejected</code>
<code>/api/zones/risk/</code>	GET	Public	Current risk score per zone (for heatmap)
<code>/api/zones/{id}/dispatch/</code>	PATCH	Admin	Update <code>dispatch_status</code>
<code>/api/route/</code>	GET	Public	<code>?from=lat,lng&to=lat,lng</code> → risk-aware safest route
<code>/api/priority-queue/</code>	GET	Admin	Zones sorted by risk score with <code>dispatch_status</code>
<code>/api/simulate-rainfall/</code>	POST	Admin	Triggers the 8-stage demo simulation (see Section 8, Step 6)

Alerts are not a user-facing endpoint — they're triggered server-side (in the risk-scoring job) whenever a zone's score crosses into "High" or "Severe," and logged in `AlertLog`.

7. Instructions for the Build Platform (Antigravity / vibe-coding)

Follow Section 8 one step at a time, in order. After each step:

1. Show what was built/changed.
2. Stop and wait for explicit confirmation before starting the next step.
3. Do not skip ahead, combine steps, or add anything not listed in that step or outside Section 2/3's scope.
4. If something is ambiguous, ask rather than guessing or expanding scope.

Keep the number of steps as low as possible — do not split these further.

8. Build Steps

Step 1 — Project scaffolding

- Django project with apps: `zones`, `reports`, `risk`, `routing`, `alerts`.
- Next.js web application (JavaScript), Tailwind CSS configured.
- Connect Django to PostgreSQL (Supabase) with PostGIS enabled.
- Running skeleton only — no business logic yet.

Step 2 — Data models + seed data

- Create all models from Section 4 with GeoDjango fields.
- Run migrations against Supabase database.
- Seed 8-10 sample Nagpur wards (approximate polygons), a few days of simulated traffic data, and pull an initial batch of real rainfall data from the IMD API.

Step 3 — Backend core logic + API

- Risk scoring function (Section 5) as a standalone module.
- Routing function (`osmnx` + `NetworkX` + A^*), weighted by current risk scores.
- Hugging Face module: sends citizen-submitted photos to Hugging Face Inference API for both pothole and waterlogging-presence detection.
- Twilio integration: sends an alert whenever a zone's computed risk crosses High/Severe, logs to `AlertLog`.
- All endpoints from Section 6, with public/admin access enforced.

Step 4 — Citizen-facing frontend

- Map view (Leaflet) showing zones colored by risk score.
- Report form: location (map click or GPS) + photo upload + description → posts to `/api/reports/`, shows Hugging Face detection result back to the citizen.
- Route search: pick start/end → calls `/api/route/` → draws the safer route.
- Mobile-responsive layout (Tailwind breakpoints).

Step 5 — Admin dashboard frontend

- Login-gated.
- Risk heatmap (admin view).
- Priority queue table: zone, risk score/category, `dispatch_status` (dropdown to update), sorted by risk.
- Reports list with verify/reject buttons.
- Alert log view (what was sent, when, to which zone).

Step 6 — Integration test, simulation mode, and deploy

- Build the rainfall simulation trigger (`/api/simulate-rainfall/`) that walks the system through: baseline → rainfall onset → risk detection → waterlogging emergence → citizen reports → alert dispatched → dispatch status updates. A single admin-facing button that drives this end-to-end for demo purposes.
- Confirm full pipeline reacts correctly: risk updates → heatmap reflects it → route avoids risk zone → priority queue reorders → alert fires → dispatch status is settable.
- Deploy: Frontend → **Netlify** (Next.js Runtime / Adapter), Backend → **Render**, Database → **Supabase** (PostgreSQL + PostGIS).

Reminder for the build agent: Stop after every step above and wait for confirmation. Build only what maps to Section 2 and the additions explicitly listed in Section 3-6. Do not add anything from the "explicitly out of scope" list in Section 2.